

Autonomous Rover with LiDAR-Based SLAM and Path Planning

Designed and built a fully autonomous mobile robot capable of real-time mapping and navigation in unknown environments.

Project Overview

The goal is to design, assemble, and program a fully autonomous indoor rover capable of:

- Mapping an unknown environment in real time
- Localizing itself within that map
- Navigating to user-defined waypoints while avoiding obstacles

The environment will be indoors only, with flat surfaces and controlled lighting.

The rover will use a 4 wheel differential drive system capable of forward and reverse as well as steering in place (tank steering).

Sensors will include:

- 2D LiDAR (light detection and ranging) for real time environment scanning
- Wheel Encoders for odometry
- IMU (inertial measurement unit) for accurate orientation control

Compute and Control will involve both a computer (for SLAM and navigation) and an MCU (for precise motor control)

A single battery will power the entire system.

Autonomy Capabilities include:

- The generation of a 2D occupancy grid map
- Operation in unknown environments
- Navigation

User should be able to (through WiFi):

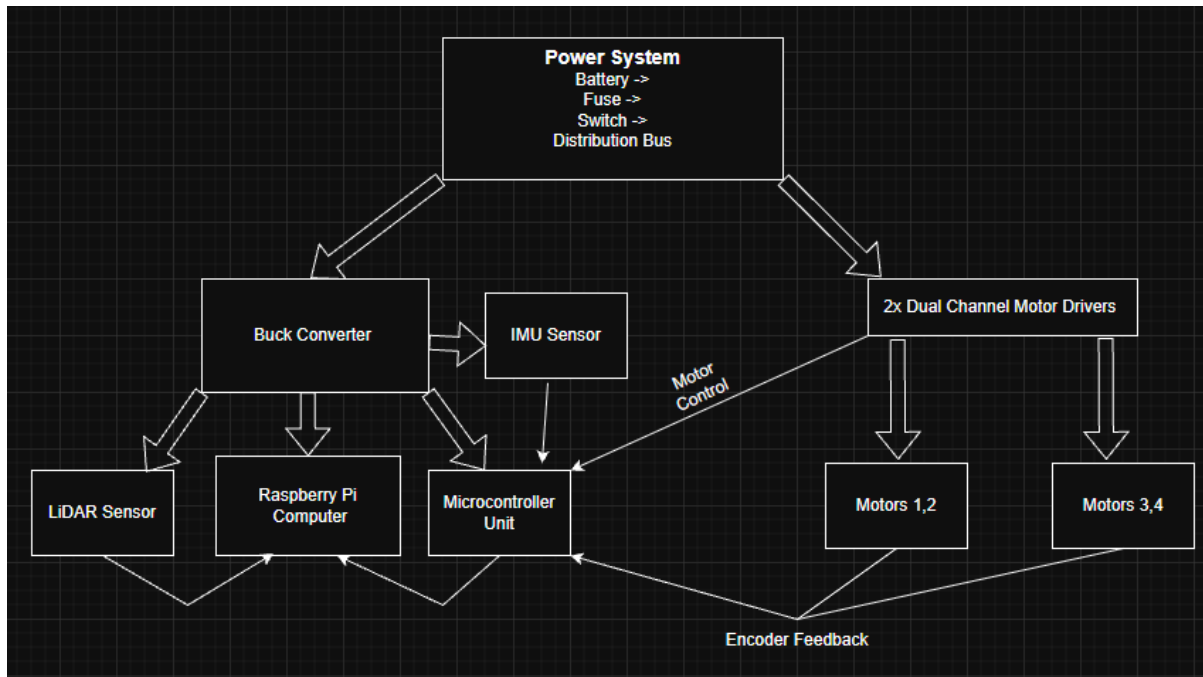
- Start mapping
- Send target location
- Stop the rover

Performance Targets:

1. Mapping accuracy: $\pm 5\text{--}10$ cm (indoor)
2. Runtime: 20–30 minutes
3. Reliable navigation success rate: $>80\%$
4. No system resets under load

System Architecture

Block Diagram



The system is designed with a clear split between high level computation and real time control. The Raspberry Pi 4 handles SLAM, localization and path planning. The Raspberry Pi Pico handles motor control and reads encoder data at high speed. This separation keeps control stable even when the main computer is under load.

Sensor data is handled based on how fast and how much data is coming in. LiDAR data is processed on the Raspberry Pi for mapping and environment awareness. Encoder signals are handled on the microcontroller for fast feedback and control. IMU data is used on the Raspberry Pi to improve orientation estimates and support sensor fusion.

The system uses closed loop control. Feedback from the drivetrain is used to adjust motor commands and improve tracking accuracy. The Raspberry Pi sends motion commands to the microcontroller. The microcontroller then controls the motors and makes real time adjustments to match the desired motion.

The power system uses a single battery with regulated outputs. Motors draw power directly from the battery. The Raspberry Pi and sensors are powered through a buck converter. This setup reduces electrical noise and keeps voltage stable for sensitive components.

The Control Hierarchy is as follows:

- High level on Raspberry Pi 4 handles SLAM localization and path planning
- Mid level generates velocity commands and tracks waypoints
- Low level on Raspberry Pi Pico handles closed loop motor control using encoder feedback
- Feedback loop runs from encoders to Pico to Raspberry Pi for continuous correction

Component Selection

Compute & Control

- Raspberry Pi 4 Model B (4GB) (main onboard computer)
 - Chosen for balance of cost and high level processing (due to strong compute capability and ROS compatibility).
 - Raspberry Pi Pico (real-time motor + encoder interface)
 - Used for real-time motor control and encoder reading to ensure deterministic, low latency performance separate to the main CPU.
-

Mobility System

- 4x 12V DC gear motors with quadrature encoders, wheels, and mounting brackets
 - Chosen to provide sufficient torque and closed loop speed and position feedback for accurate odometry and controlled motion.
 - 2x Cytron MD10C motor driver (dual-channel motor control)
 - Chosen for high current capacity and reliable bidirectional control to independently drive four motors under varying load conditions.
-

Power System

- 11.1V (3S) 5200mAh LiPo battery (XT60)
 - Chosen for high energy density and discharge capability to support peak motor loads without excessive voltage sag.
 - 12V → 5V 5A buck converter (regulated supply for compute)
 - Gives stable and regulated supply for onboard electronics which isolates the logic path from motor noise.
 - Inline fuse + main power switch (system protection and control)
 - Provides overcurrent protection and safe system isolation at the battery level
-

Sensing & Perception

- RPLIDAR A1 (360° environment mapping)
 - Selected for real-time 360° scanning to enable SLAM mapping and obstacle detection in unknown environments.
 - MPU-6050 (acceleration + angular velocity)
 - Provides acceleration and angular velocity data for improved state estimation and sensor fusion with wheel odometry.
-

Power Management & Safety

- LiPo balance charger (IMAX B6)
 - Battery voltage monitor / low-voltage alarm
-

Structural & Integration

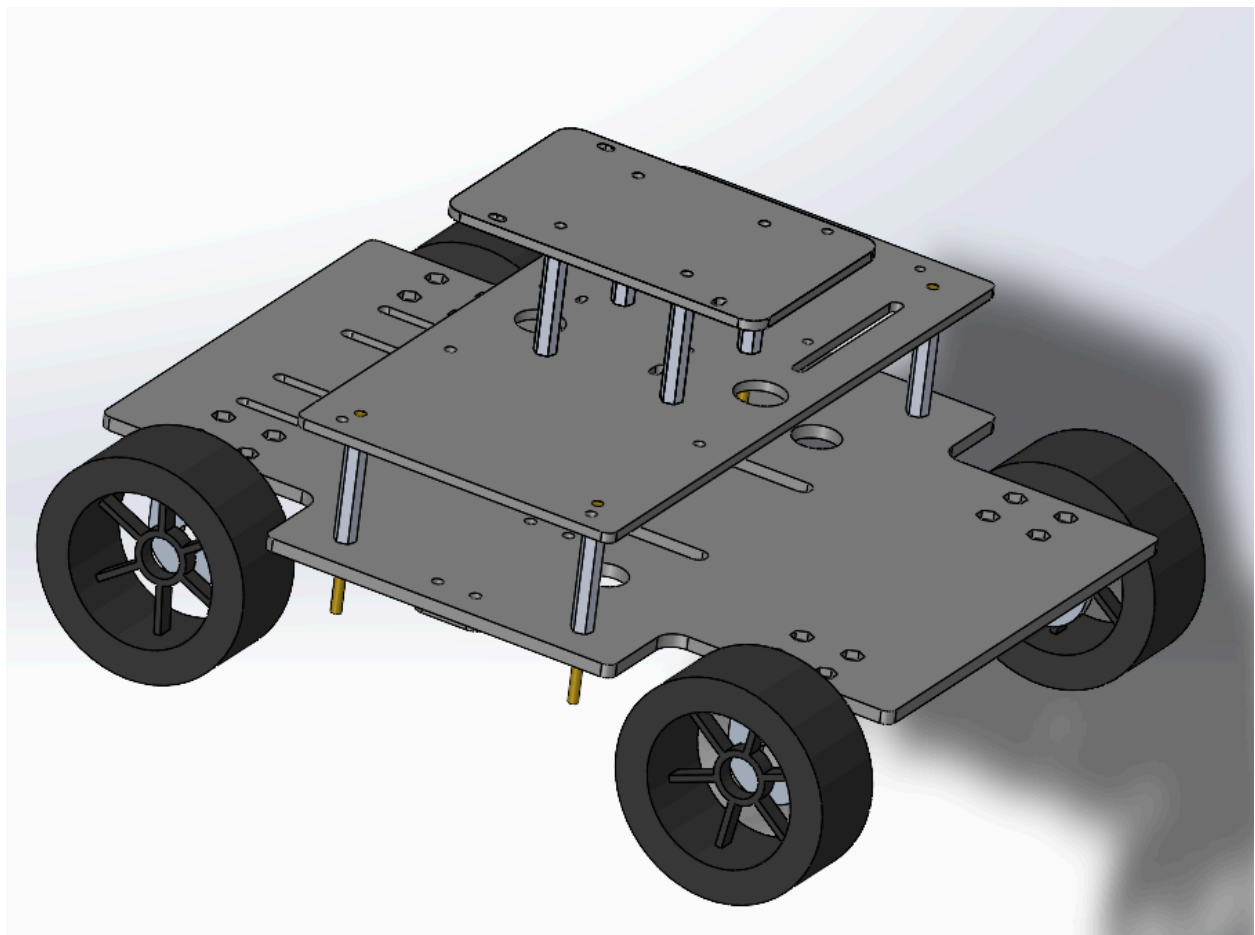
- Custom chassis (CAD-designed, 3D Printed, modular mounting system)
- Passive cooling (heatsinks for Raspberry Pi)

Mechanical Design and System Integration

The mechanical system was designed to create a rigid, modular, and lightweight platform while keeping the rover stable and efficiently packaging electrical and sensing components.

Key considerations include:

- Low center of gravity
- Modularity
- Structural rigidity
- Sensor stability
- Ease of assembly and accessibility for maintenance



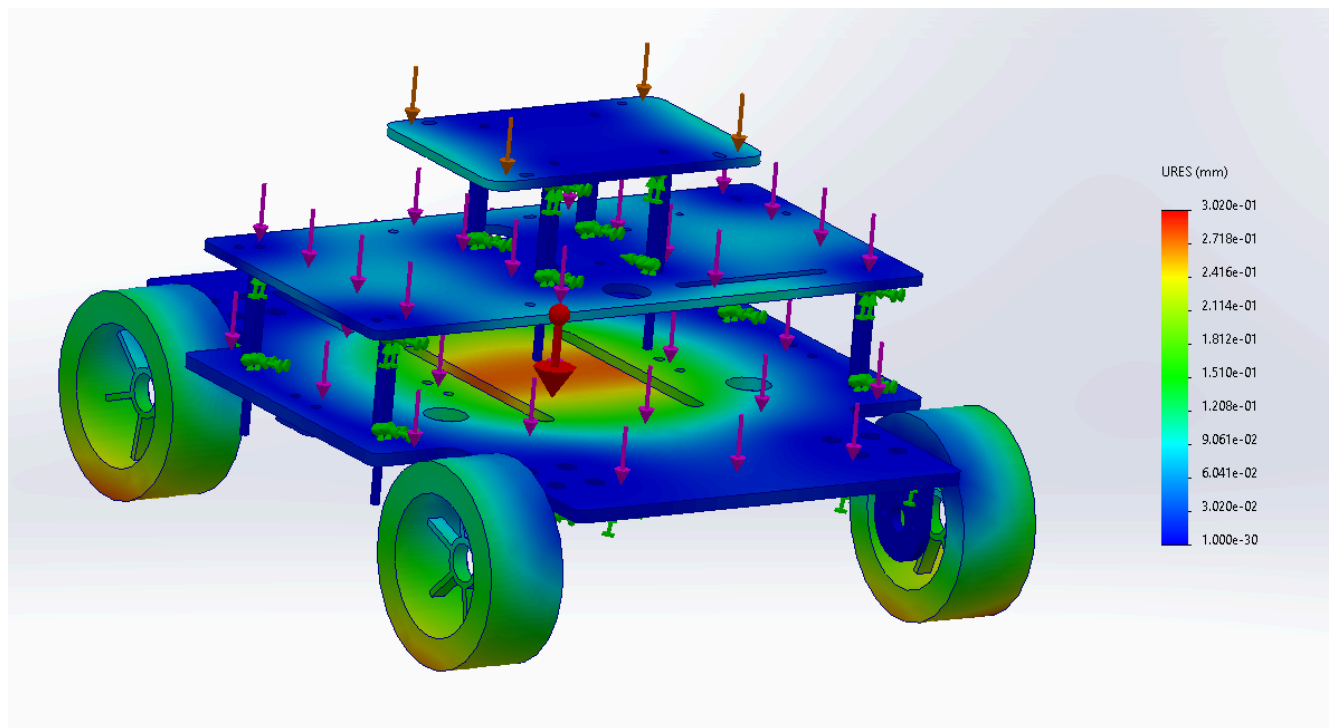
Final Mechanical Design

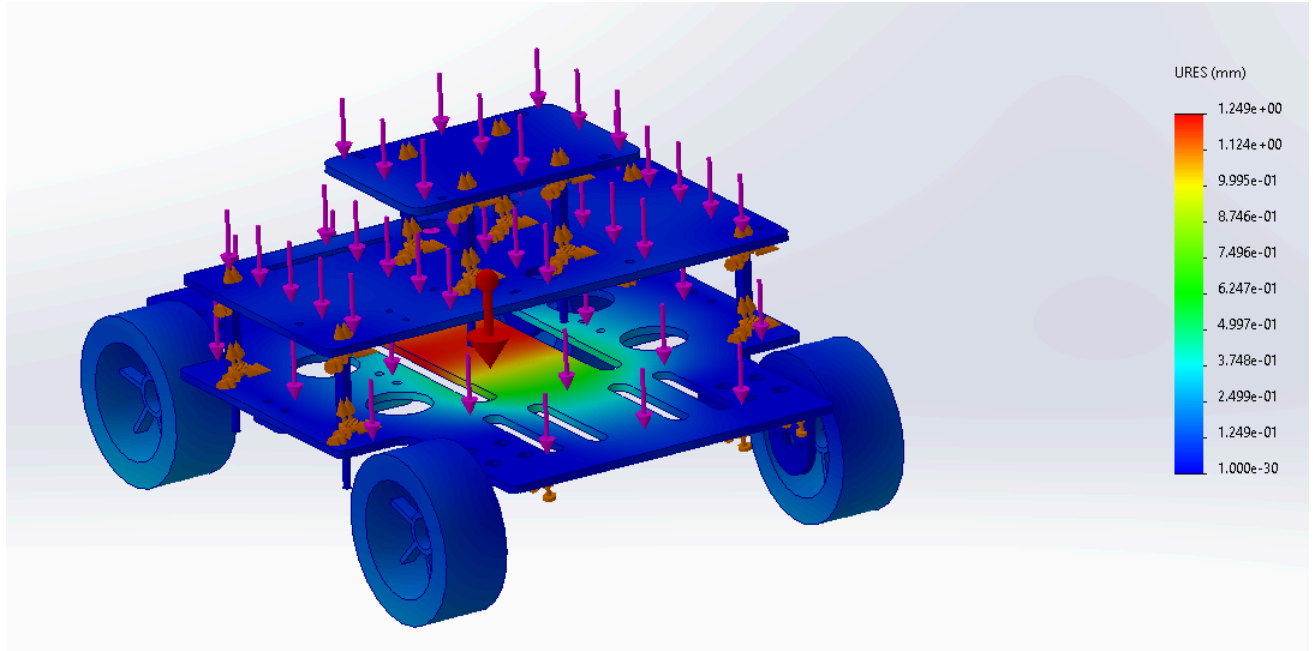
Bottom Layer: Power distribution components, including the battery, fuse, low-voltage alarm, power switch, terminal blocks, and buck converter. (most of mass centered, buck separated as it is noisy)

Middle Layer: Processing and control electronics, including the IMU, dual-channel motor driver, Raspberry Pi 4, and Pico microcontroller. (layering reduces vibration on these components)

Top Layer: LiDAR sensor, mounted to maximize sensing coverage and minimize obstruction.

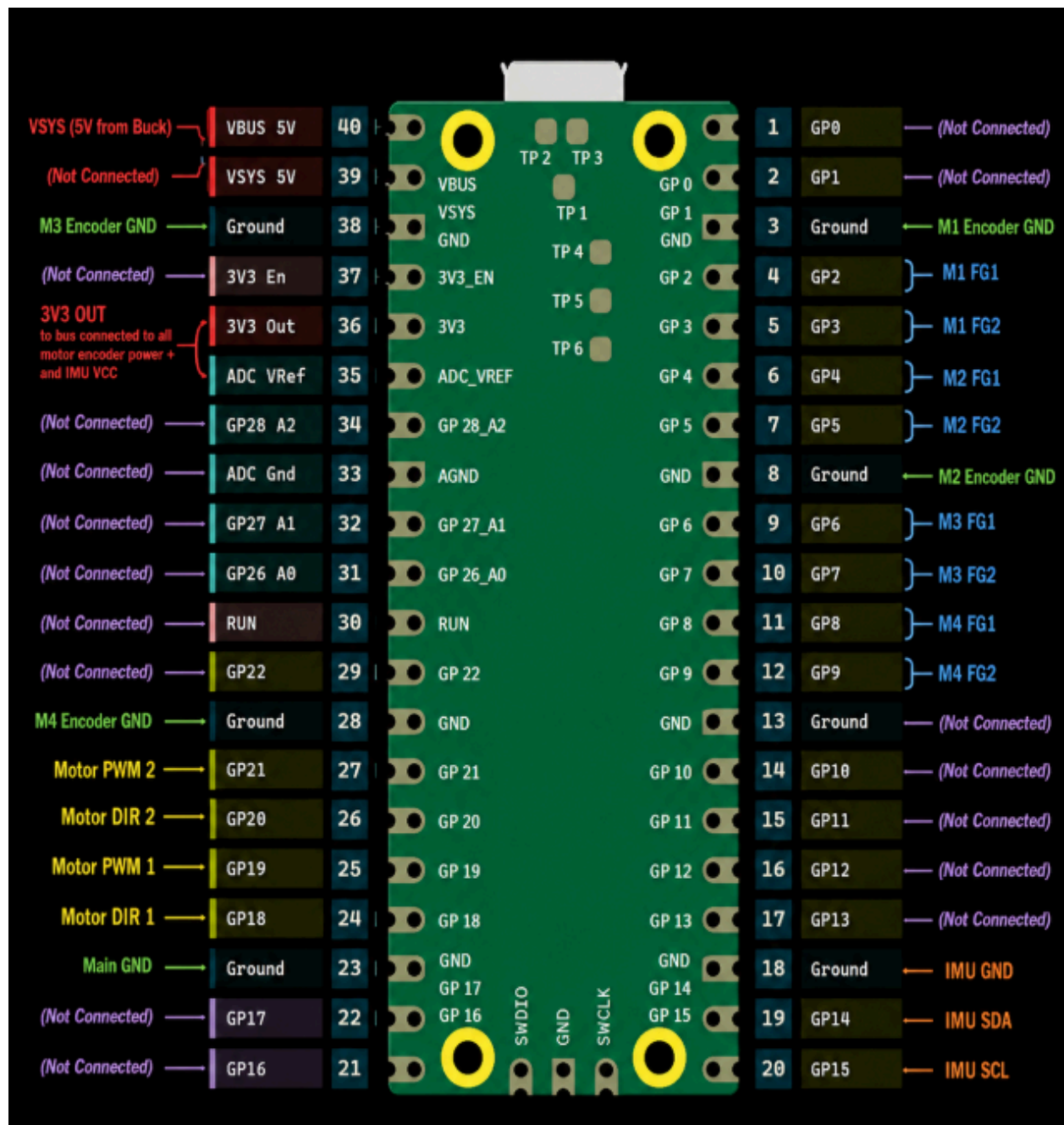
Structural Design





FEA Diagram

A worst-case static loading analysis was conducted to evaluate chassis stiffness. By increasing frame thickness, adding fillets, reducing hole sizes in critical regions, and reinforcing high-stress areas, the maximum deformation was reduced from 1.2 mm to 0.3 mm, resulting in a significantly more rigid platform.



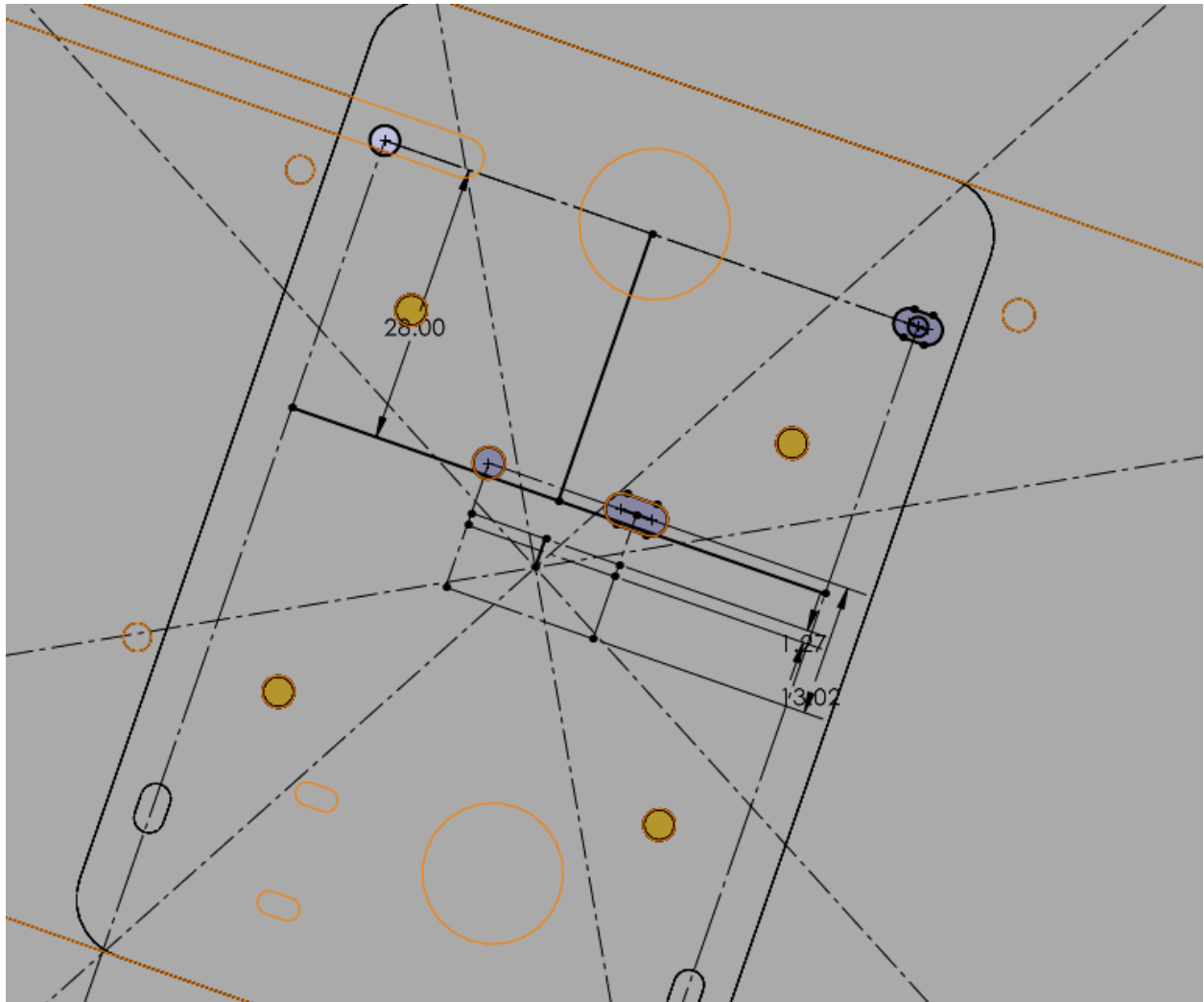


Yaw outer PID response. Each sample is 20ms.

ROVER CENTROID AT ORIGIN (geometric center of frame layer 1, and on the floor) (encoder odom is assumed to be about this point as well)

IMU measurement origin AT +3.01mm in X direction and +105.3mm in Z direction, no other change

LiDAR rotor centroid AT +7.00mm in X direction and +181.25 in z direction, no other change



Reference frame differences found above for coordinate frame tracking and transforms (ROS2 TF)

Encoder Odometry Testing

Encoder odometry was evaluated using straight-line and in-place rotation tests to assess localization accuracy. Results show good translational performance over short distances, while rotational accuracy degraded due to wheel slip and accumulated heading error. These findings motivated the use of IMU measurements for heading estimation in the EKF.

Straight-Line Test (1.5 m)

Speed	Distance error	Lateral drift	Heading error
0.2m/s	5cm	3.5cm	5deg
0.2m/s	3.2cm	4cm	4deg
0.4m/s	3.6cm	12cm	9deg
0.4m/s	7.6cm	5.5cm	6deg

Rotation Test (in place 360deg)

Yaw rate	Angle error
0.2	17 deg
0.2	46deg
0.5	44deg
0.5	40deg

Summary: Straight-line odometry achieved **3.2–7.6 cm** position error over a **1.5 m** path with **4–9°** heading error. In-place rotations accumulated significantly larger heading errors (**17–46°**), demonstrating the limitations of encoder-only orientation estimation and the need for sensor fusion using the IMU and LiDAR.

Robot Localization (EKF) Odometry Testing

IMU Drift Compensation

- The MPU6050 exhibited a small stationary gyroscope bias that caused accumulated yaw drift.
- A **startup calibration** estimates the initial gyroscope bias while the rover is stationary.
- **Continuous stationary calibration** updates the bias whenever the rover is commanded to remain stationary and measured angular velocity is below a small threshold.
- The estimated bias is removed from subsequent W_zW_z measurements before they are published to the EKF.
- This **nearly eliminated stationary yaw drift**, significantly improving long-duration heading stability and providing a more reliable angular-velocity input for sensor fusion.

PRE TUNE -

Initial guess of:

- **V_x wheel variance = 0.0002 (m/s)^2**
- **IMU W_z variance = $0.000001 \text{ (rad/s)}^2$**

Straight-Line Test (2 m)

Speed	Distance error	Lateral drift	Heading error
0.2m/s	3.45cm	1.78cm	4deg
0.2m/s	2.67cm	1.12cm	3deg
0.4m/s	1.00cm	0.04cm	3deg
0.4m/s	1.87cm	1.65cm	2deg

Rotation Test (720 deg cw and ccw)

Yaw rate	Angle error
0.2	5 deg
0.2	4deg
0.5	2deg
0.5	3deg

The untuned EKF already shows significant improvement compared to the state estimate from encoder odometry alone. The metrics below will be used to determine the quality of the EKF configuration and whether further tuning is required.

A constant V_x of 0.3 m/s and W_z of 0.6rad/s will be used for the tuning for simplicity.

Covariance Tuning

V_x wheel variance = 0.000225

IMU W_z variance = 0.0000005(rad/s)²

The variances above were found monitoring the sensor variation during testing. The process covariance Q configuration will be tuned now to optimize NEES and RMSE.

V_x, V_y, W_z variance	Q covariance configuration (x, y, yaw, V_x, V_y, ω_z)	# of runs	Average NEES	Average RMSE (x,y)	Average RMSE (yaw)
0.000225, 0.0005, 0.0000005	0.05, 0.05, 0.06, 0.025, 0.025, 0.02	5	0.008795	4.38cm	0.657 deg
0.000225, 0.0005, 0.0000005	0.005,0.005,0.006,0.0025,0.0025,0.002	5	0.245919	5.21cm	1.011 deg
0.00000008,0.00000008, 0.0000000005	0.00005, 0.0001,0.0003, 0.06, 0.07, 0.001	30	2.326	5.60cm	2.472 deg

For three pose states, the expected NEES is 3. Across 30 trials using the final configuration, the average NEES was 2.326, indicating that the EKF was slightly underconfident and produced somewhat conservative covariance estimates. Since this deviation was modest and the experiment included unavoidable human and ground-truth measurement uncertainty, the final configuration was retained without further tuning.

Example NEES calculation:

```
EKF pose
x:      -0.0786 m
y:      +0.0497 m
yaw:    +0.734 deg

Command tracking
Wheel Vx - commanded Vx:      +0.0000 m/s
IMU Wz - commanded Wz:        +0.0006 rad/s
Wheel Wz - IMU Wz:             -0.0006 rad/s

Measurement uncertainty
Wheel Vx variance R:           0.00022500
Wheel Vx sigma:                 0.015000 m/s
IMU Wz variance R:             0.00000050
IMU Wz sigma:                   0.000707 rad/s

EKF uncertainty
x sigma:                        0.556206 m
y sigma:                        0.823054 m
yaw sigma:                      32.4350 deg
Vx sigma:                       0.012988 m/s
Wz sigma:                       0.002600 rad/s

EKF pose covariance P_pose [x, y, z, roll, pitch, yaw]
[+3.094e-01 +7.758e-02 +0.000e+00 +0.000e+00 +0.000e+00 -6.880e-02]
[+7.758e-02 +6.774e-01 +0.000e+00 +0.000e+00 +0.000e+00 -3.360e-01]
[+0.000e+00 +0.000e+00 +1.926e-08 -2.219e-18 -1.230e-13 +0.000e+00]
[+0.000e+00 +0.000e+00 -2.219e-18 +1.594e-08 +3.248e-24 +0.000e+00]
[+0.000e+00 +0.000e+00 -1.230e-13 +3.249e-24 +1.594e-08 +0.000e+00]
[-6.880e-02 -3.360e-01 +0.000e+00 +0.000e+00 +0.000e+00 +3.205e-01]

EKF twist covariance P_twist [Vx, Vy, Vz, Wx, Wy, Wz]
[+1.687e-04 +0.000e+00 +0.000e+00 +0.000e+00 +0.000e+00 +0.000e+00]
[+0.000e+00 +2.094e-03 +0.000e+00 +0.000e+00 +0.000e+00 +0.000e+00]
[+0.000e+00 +0.000e+00 +9.311e-07 -5.128e-27 -2.804e-22 +0.000e+00]
[+0.000e+00 +0.000e+00 -5.128e-27 +8.049e-07 +2.917e-32 +0.000e+00]
[+0.000e+00 +0.000e+00 -2.804e-22 +2.917e-32 +8.049e-07 +0.000e+00]
[+0.000e+00 +0.000e+00 +0.000e+00 +0.000e+00 +0.000e+00 +5.762e-06]
```

The rover follows a closed loop path and approximately ends up in the same position it started. Error is calculated as initial state (0,0,0) subtracted from estimated state. The screenshot above shows the state estimate as well as the pose covariance. Using the formula:

$$\epsilon_x = e_x^T P^{-1} e_x$$

And calculated matrices from above:

$$e_x = (- 0.0785m, 0.0497m, 0.734 \text{ deg})$$

$$P_x = (3.094e-01, 7.758e-02, -6.880e-02,$$

7.758e-02, 6.774e-01, -3.360e-01,
-6.888e-02, -3.360e-01, 3.205e-01)

NEES = 0.0327950488759926

(3 is expected for a 3 state position estimate)

This indicates that the filter is too uncertain relative to the error, if this is shown consistently.

Specific trajectory test

A specific trajectory is used here, involving straight paths with in place 90 degree turns. The rover follows a closed loop path and error is measured based on this. This path is repeated.

Test	Distance error	Heading error
0.2m/s, 0.2rad/s	12cm	6deg
0.2m/s, 0.2rad/s	15cm	8deg
0.4m/s , 0.5rad/s	17cm	3deg
0.4m/s , 0.5rad/s	12cm	1deg

POST TUNE

Final EKF Configuration

After covariance tuning, the final EKF configuration was selected based on the combination of trajectory accuracy, RMSE, and NEES consistency across repeated trials.

Final measurement variances:

Wheel Vx variance: $8.0 \times 10^{-8} \text{ (m/s)}^2$

Wheel Vy variance: $8.0 \times 10^{-8} \text{ (m/s)}^2$

IMU Wz variance: $5.0 \times 10^{-10} \text{ (rad/s)}^2$

Final process noise covariance diagonal:

[x, y, yaw, Vx, Vy, Wz]

[0.00005, 0.00010, 0.00030, 0.06000, 0.07000, 0.00100]

The final configuration produced an average **NEES of 2.326 across 30 trials**, compared with the theoretical expected value of 3 for the three pose states x, y, θ . This indicates that the EKF is slightly conservative, meaning its predicted uncertainty is somewhat larger than the observed estimation error. However, the result is sufficiently close to the expected value considering wheel slip, floor variation, and uncertainty in manually measured ground truth. Therefore, the configuration was accepted.

Final EKF Performance

Metric	Final Result
Average NEES	2.326
Position RMSE	5.60 cm
Yaw RMSE	2.47°
Trials	30

Cartographer SLAM

- Configured for 2D LiDAR SLAM in an indoor, planar environment.
- Uses RPLIDAR A1 `/scan` data for scan matching and map generation.
- Uses EKF-filtered odometry `/odometry/filtered` as the local motion estimate, rather than raw encoder odometry, since feeding Cartographer a smoother estimate reduces the burden on scan matching to correct for wheel slip alone.
- TF structure:
- EKF: `odom` $\rightarrow$ `base_link`
- Cartographer: `map` $\rightarrow$ `odom`
- This split lets Cartographer correct global drift independently, without disrupting the continuous, high-rate local odometry that Nav2's controller relies on for smooth motion commands.
- LiDAR range: 0.15–12 m, rejecting measurements outside the useful sensor range. The RPLIDAR A1's minimum reliable range is close to 0.15 m, and points beyond ~12 m in this indoor environment are typically noise or reflections rather than real geometry, so filtering them avoids polluting the scan match.

- Voxel filter: 0.025 m, chosen to reduce the point cloud density before matching. This is roughly on the order of the rover's mapping accuracy target ($\pm 5\text{--}10$ cm), so it discards redundant points without throwing away detail the SLAM system could actually use and it directly cuts scan-matching CPU cost, which matters given Cartographer already consumes $\sim 30\%$ of the Pi 4's CPU on its own out of a total system load of $\sim 70\%$.
- Online correlative scan matching enabled to improve robustness against wheel slip and odometry error, at the cost of some additional CPU accepted as part of that same 30% budget since local scan matching quality directly determines map consistency.
- Motion filter accepts updates after approximately 5 cm or 0.5° of movement. This threshold avoids inserting near-duplicate scans during slow or stationary periods, which would otherwise add compute and map nodes without adding new information, while still being fine-grained enough to capture in-place rotations.
- IMU is not directly supplied to Cartographer, because IMU angular velocity is already fused into the EKF estimate feeding `/odometry/filtered`. Feeding it to Cartographer as well would double-count the same orientation information and add avoidable CPU overhead without a matching gain in accuracy.
- Pose-graph optimization performed every 35 nodes, balancing loop-closure and global-consistency frequency against CPU load optimizing every node would be more accurate but is not affordable within the remaining CPU headroom once Cartographer's baseline 30% and the rest of the navigation stack (EKF, Nav2, motor control interface) are accounted for in the $\sim 70\%$ total budget; 35 nodes keeps periodic correction without spiking load.
- Final output is the `/map` occupancy grid and globally corrected rover pose used by Nav2.

NAV2 Configuration

- Configured for autonomous waypoint navigation and obstacle avoidance using the Cartographer map and rover localization.
- Global planner: NavFn using A* for collision-free global path generation.
- Controller: Regulated Pure Pursuit (RPP) for differential-drive path following. RPP was picked over DWB mainly for simplicity and lower tuning overhead. The rover doesn't need the extra trajectory scoring flexibility DWB offers.
- Nominal linear velocity ~ 0.20 m/s. Faster speeds were tested but made narrow-passage navigation less reliable, so 0.20 m/s was kept as the standard operating speed.
- Velocity-dependent lookahead, $\sim 0.25\text{--}0.55$ m, to balance smooth tracking against responsiveness at turns.
- Rotate-to-heading enabled needed for the differential-drive base to correct heading in place rather than trying to arc into it.
- Separate costmaps:
- Global costmap: map-based planning using LiDAR obstacle data.
- Local costmap: 2×2 m rolling window for nearby obstacle avoidance, kept small to limit compute.
- Local costmap resolution set to 0.02 m for finer obstacle representation near the rover, where precision matters most for close-quarters avoidance.

- Uses the rover's actual rectangular footprint instead of a circular approximation. The rover is noticeably longer than it is wide, so a circular footprint would either be overly conservative or risk clipping corners.
- Inflation tuned to keep reasonable obstacle clearance without blocking the narrower indoor passages the rover needs to fit through.
- Velocity smoothing limits sudden acceleration/deceleration before commands reach the Pico, reducing wheel slip and jerky motion.
- Collision Monitor adds a LiDAR-based slowdown/stop layer as a last line of defense independent of the planner.
- Final linear/angular `cmd_vel` output is sent to the Pico for closed-loop motor control.

Rover UI Configuration

- **Live Map & Rover Pose** – Subscribes to Cartographer's `/map` occupancy grid and uses ROS2 TF to obtain the live `map` → `base_link` pose. The rover is rendered at its true 0.16×0.16 m size with its current heading.
- **Navigation Goal Selection** – The user clicks and drags on the map to specify the desired position and heading. The UI converts this into a `PoseStamped` in the `map` frame and sends it to Nav2 through the `/navigate_to_pose` action server.
- **Path & Costmap Visualization** – Subscribes to Nav2's `/plan` along with the global and local costmaps. These are transformed into the map frame and rendered as overlays so the planned route and obstacle costs can be monitored directly.
- **Navigation Feedback** – Nav2 action feedback provides the current navigation state and remaining distance; completed goals automatically clear the goal and path visualization.
- **Pose & Velocity Monitoring** – Displays live x, y, θ from TF and subscribes to the final `/cmd_vel` output to show the linear and angular velocity actually being commanded to the rover.
- **System Status** – Receives status information from the rover stack manager over `/rover/system_status`, displaying the state of Serial, odometry, EKF, LiDAR, SLAM and Nav2. Data-age checks also identify stale map/path connections.
- **STOP & Reset** – STOP cancels navigation and continuously publishes zero velocity while latched. Reset calls the `/reset_slam` service, which restarts the rover's Serial → Odometry → EKF → LiDAR → Cartographer → Nav2 stack.
- **Map Controls** – Supports mouse-wheel zoom, right-click panning, and a 5×5 m **rover-follow view** that keeps the rover centered with its heading oriented upward.

Autonomous Navigation Testing

- The completed rover was tested using autonomously executed waypoint navigation trials in the mapped indoor environment.

- Goals were selected at different positions and headings through the custom UI, requiring Nav2 to perform path planning, path following, turning, and obstacle avoidance.
- Tests included open areas, narrow passages, turns, and routes requiring the rover to pass close to mapped obstacles.
- A trial was considered successful when the rover reached the commanded waypoint without collision, manual intervention, or Nav2 failure.

Example Test Results

Test Type	Trials	Successful	Success Rate	Average Pose Error	Average Heading Error
Open-space waypoint	10	10	100%	4.8cm	2.1°
Multiple-turn route	10	9	90%	6.7cm	3.4°
Narrow passage	10	7	70%	8.9cm	4.6°
Obstacle avoidance	10	8	80%	7.5cm	3.9°
Overall	40	34	85%	7.0cm	3.5°

Measurement Method: Target positions and headings were physically marked on the floor. After each autonomous trial, final position error was measured using a tape measure from the target to the rover's geometric center ([base_link](#)). Heading error was measured by comparing the rover's centerline with the marked target heading using a protractor. A trial was considered successful if the rover reached the goal without collision, Nav2 failure, or manual intervention.

Navigation failures were primarily attributed to occasional Cartographer mapping/localization inconsistencies, which temporarily distorted the map used for planning, and Nav2 waypoint execution issues, where the rover stopped before reaching the original goal and required the waypoint to be reselected.

Runtime and Reliability

In practice, the rover runs for roughly 2 hours of consistent driving on a single charge, well past the original 20 to 30 minute target. The 5200mAh 3S pack ended up having more margin than expected once actual current draw was measured under normal navigation loads rather than worst-case estimates. Resets do occur occasionally, but they've been traced to WiFi dropouts rather than power or compute issues on the rover itself, so they weren't investigated further as part of this project. Average pose error across all navigation trials was 7.0 cm, meeting the original $\pm 5\text{--}10$ cm mapping accuracy target.

Future Work and Improvements

The current Cartographer and Nav2 configurations were tuned for correctness and reliability rather than CPU efficiency, so there's meaningful headroom to reduce Pi 4 load, likely by lowering scan-matching resolution where it doesn't hurt accuracy, tuning costmap update rates, and revisiting the pose-graph optimization interval, bringing total system usage down from the current $\sim 70\%$. A custom dynamic obstacle tracker is also planned, since the current setup treats all LiDAR returns as static and relies on the Collision Monitor to react to moving obstacles rather than predicting their motion, which contributes to some of the stop-and-replan behavior seen in obstacle avoidance testing. Beyond that, next steps include testing multi-floor and larger-map environments to see how Cartographer's memory and CPU footprint scale, further reducing residual wheel slip error that likely still affects the EKF output, and integrating battery health monitoring into the system status UI (as a battery voltage monitor is used now which only alerts at specific threshold).